

Property-Based Testing for Mobile UI

Finding harder bugs earlier

Hey, I'm PJ

- Platform stuff @ OkCredit(YC S18)
- I'm here to talk about tough bugs

Let's talk testing

We write unit tests

```
@Test fun `balance of a single $5 debit is -$5`() {  
    val txns = listOf(Transaction(type = debit, amount = 500))  
    assertEquals(-500, balanceOf(txns))  
}
```

We write integration tests

```
@Test
fun `submitting a transaction updates balance`() {
    /*
    1. open the app, log in
    2. tap an account
    3. tap "Add Transaction"
    4. type 50, tap Submit
    5. go back home
    */

    assertEquals(Balance(5000), homeScreen.totalBalance())
}
```

All tests pass



But user reported a bug



**There are duplicate
transactions,
same amount, same time.**

**Why? We wrote tests for
this and they passed????**

The problem

Example tests cover the paths
we thought of.

Bugs live in the paths
we didn't cover.

Property-Based Testing

- Write invariant properties for testing, not just examples
- The machine generates 100s of random actions. Invariant has to hold for all of them

Property-Based Testing

EXAMPLE TEST

"When I type 50 and tap Submit, balance becomes \$50"

PROPERTY

"Transaction submit moves the balance exactly by the typed amount"

Sanderling

- Autonomous testing based on specifications
- Explore application on device and finds invalid behaviors
- Works directly on the device independent of application framework

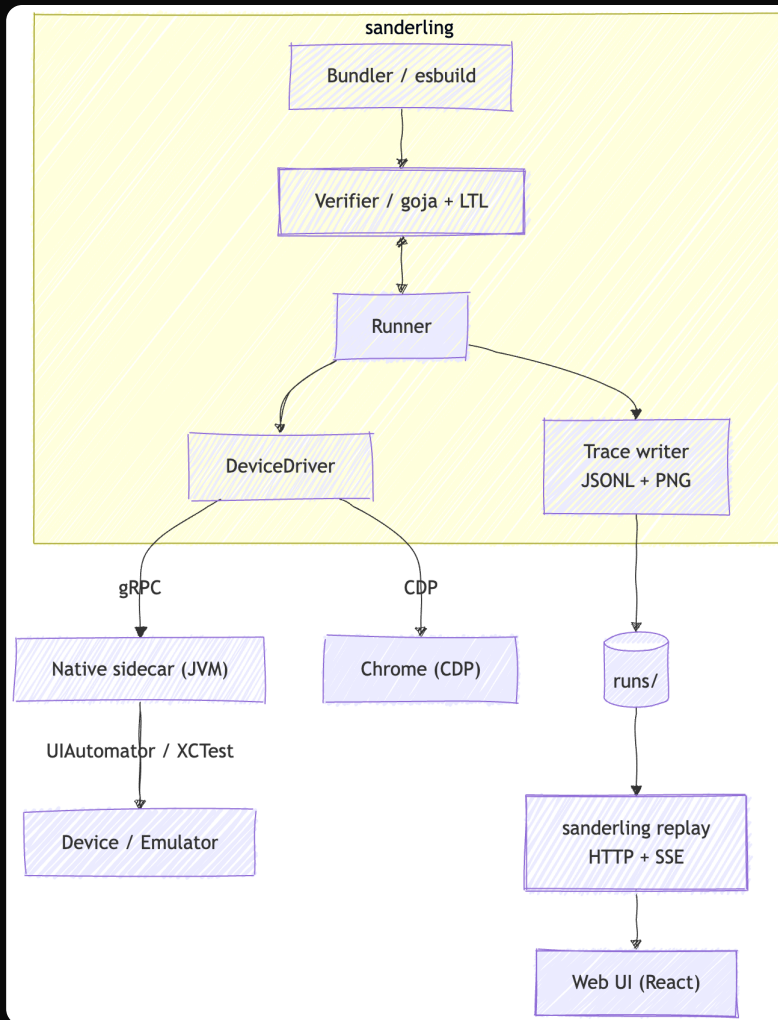
Sanderling

Specifying and validating invariant properties

```
// property definition in spec.ts
const submitMovesBalanceByTypedAmount = always(
  next() => {
    const typedAmount = parseTypedAmount(txnAmountField.previous?.text);

    const on = JSON.stringify(lastAction.on);
    if (!on.includes("TxnSubmit")) return true;
    if (typedAmount === 0) return true;

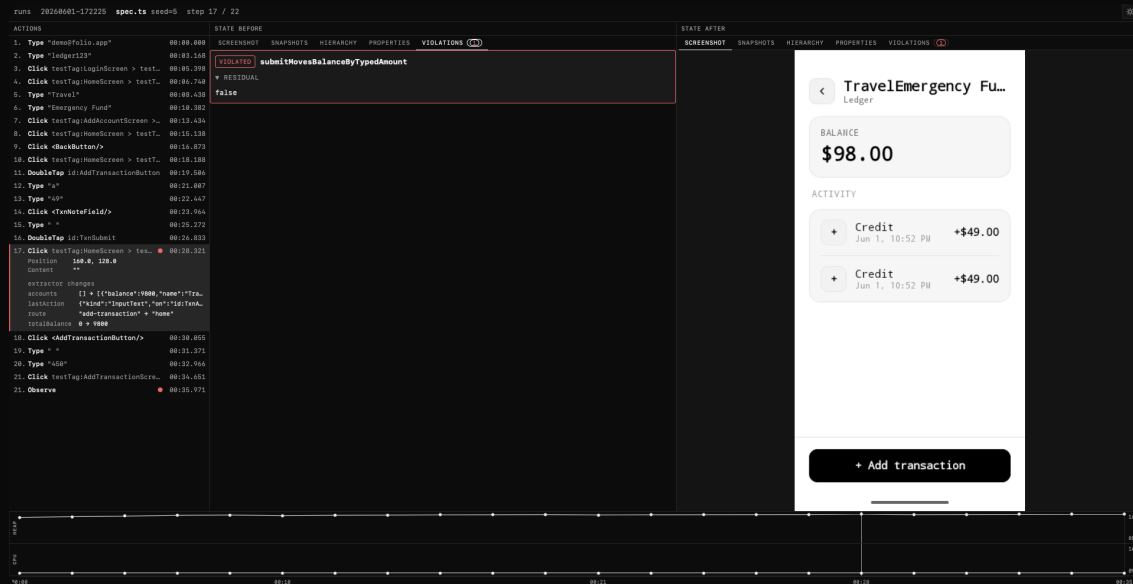
    return Math.abs(totalBalance.current - totalBalance.previous) === typed
  })
```



How it works

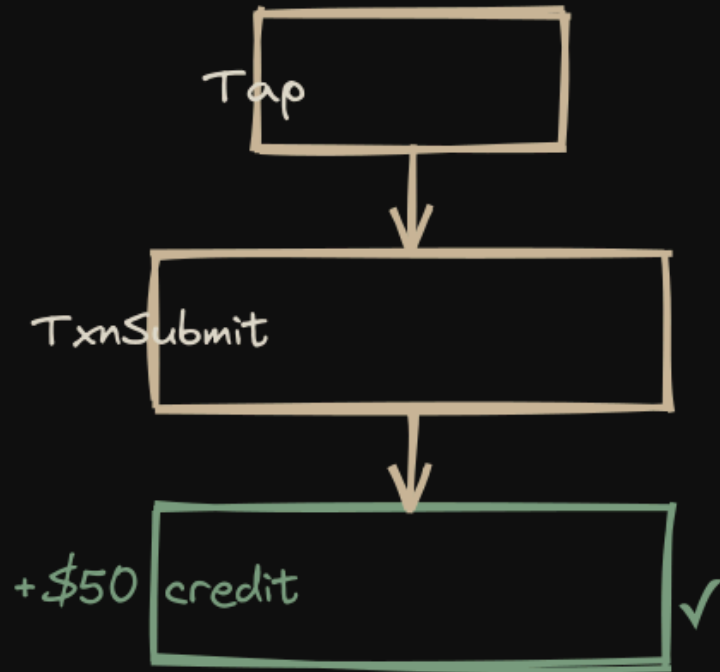
1. Extracts the current state from device
2. Checks all properties against the current state, recording violations
3. Selects the next action based on the current state using a fuzzer, and performs it on device
4. Repeats the process

When we run it for our spec

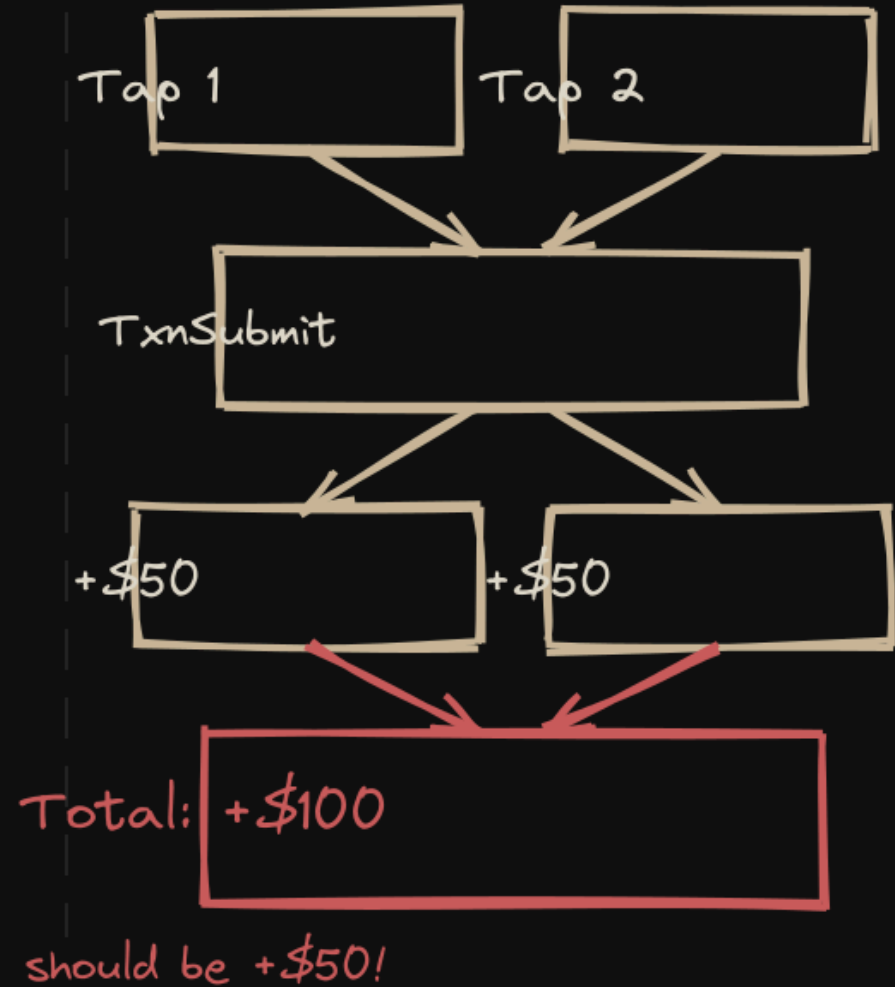


Found the bug

you tested



fuzzer found



With example test

```
@Test
fun `adding $50 credit increases balance by $50`() {
    // type 50
    // tap Submit ← just once, obviously
    // go back home

    assertEquals(5000, home.totalBalanceCents())
}
```

passed

We never thought to tap twice. The test didn't either.

With property test

```
Step 14  Tap      → AddTransactionButton
Step 15  InputText → TxnAmountField  "49"
Step 16  Tap      → TxnSubmit
Step 17  Tap      → TxnSubmit
```

```
Property violated: submitMovesBalanceByTypedAmount
  expected balance change: 4900 cents
  actual   balance change: 9800 cents ← 2x the amount
```

violation

Step 17 was random. We never wrote it.

Recap

- Example-based testing is limited with action space
- Property-based testing lets us specify invariants that must hold for randomized action space

Resources

- It's Open source github.com/priyanshujain/sanderling



If you want to use this or contribute, please
come talk to me.

